

6. Process

6.1 Project Organisation and Tools

The project is split across three Git repositories, each with its own Jenkins CI/CD pipeline deploying to AWS EC2 via Docker:

Repository	Role
flask-app	REST API (Flask/Gunicorn), JWT auth, ML predictions, journey planning, AI chat. Owns the MySQL schema via Flask-Migrate.
react-app	Single-page frontend (React 19, TypeScript, Vite 7, Tailwind CSS 4). Served by nginx in production.
scraper	Long-running Python process: polls JCDecaux stations and OpenWeatherMap forecasts, writing to the shared database.

Tooling: Git feature-branch workflow • Jenkins (Kubernetes agent) • Docker • AWS EC2 • pytest ($\approx 97\%$ coverage) • ESLint / TypeScript • Discord (team communication).

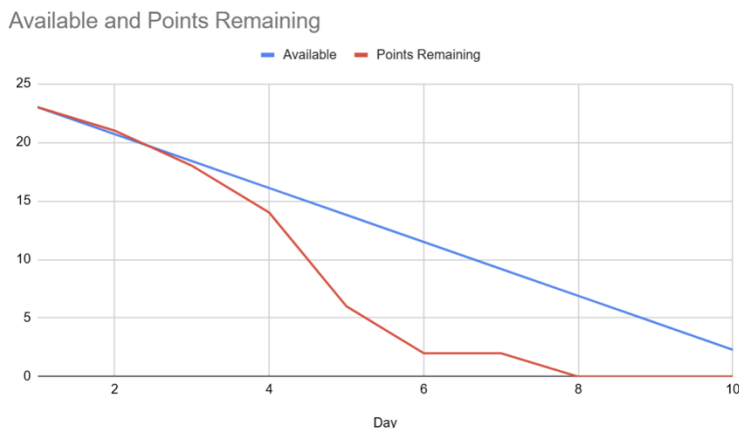
Integration contract: Both flask-app and scraper share one DATABASE_URL. Flask-Migrate migrations must run first. In development, Vite proxies /api to Flask on port 5000; in production a reverse proxy handles the same routing.

6.2 Sprint 1 — Requirements Engineering & Data Collection

Implemented Features and Design Decisions

The team conducted user interviews to validate three personas (Student, Professional, Tourist), produced Figma UI mockups, and defined user stories. On the technical side, a MySQL database schema was established and scraping scripts for the JCDecaux and OpenWeatherMap APIs were written and deployed. A 5-minute scrape interval was chosen as the design default to balance data freshness against API rate limits. The scraper ran stably throughout the sprint, consistently populating the database and providing a reliable foundation for subsequent development.

Burndown Chart



The Burndown chart shows a steep decline in remaining effort. Our "Actual Remaining Effort" line dropped below the "Ideal Trend" line around Day 4, indicating that the team worked faster than anticipated. We reached zero remaining tasks 2 days before the sprint deadline. This high velocity allowed us to dedicate the remaining time to backlog refinement and researching the feasibility of optional features.

Sprint Review

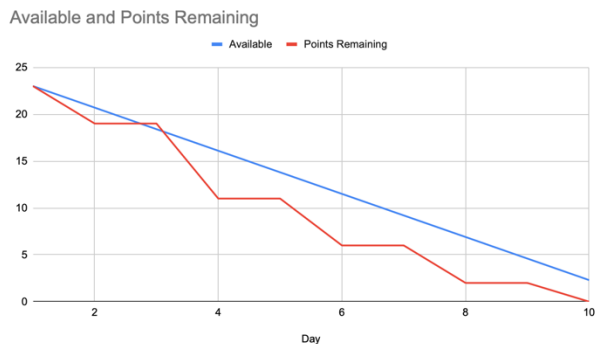
1. **Mockup Walkthrough:** Presented the Figma/PDF mockups, explaining how the UI solves the specific pain points of our personas.
2. **Data Verification:** Showed the live MySQL database tables with rows of real-time data collected over the past few days, proving the stability of our scrapers.
3. **Scope Discussion:** Since we finished early, we raised a discussion regarding the "Smart Journey Planner" feature. We are currently evaluating if adding this complex feature fits within our remaining timeline. We agreed to finalize this decision in the next Sprint Planning.

6.3 Sprint 2 — Backend Integration & Deployment Pipeline

Implemented Features and Design Decisions

The Flask application was built out with Weather and Station Read APIs returning properly formatted JSON from the shared MySQL database. JWT-based user authentication was integrated with secure password hashing. A Jenkins CI/CD pipeline was configured with a Kubernetes agent, automating Docker builds and EC2 deployments on every push to main. A Git feature-branch workflow was adopted team-wide. Crucially, the Journey Planner backend endpoint was delivered, closing the open scope question from Sprint 1, though it required significant refactoring to handle edge cases and correct route-calculation logic.

Burndown Chart



The burndown chart displays a staircase pattern which directly reflects the project requirement to update the chart every two days. Despite these mandated reporting intervals, the actual remaining effort dropped below the ideal available trend line by Day 4 and remained beneath it for the rest duration of the sprint. This demonstrates that a strong development velocity was maintained by the team. All sprint goals were completed by Day 10.

Sprint Review

1. **API Demonstration:** Showcased the operational Weather and Station Read APIs successfully returning properly formatted JSON data from both live external APIs and the local MySQL database.
2. **Security & Authentication:** Demonstrated the user authentication flow, highlighting the successful generation of JWT tokens upon login and the secure hashing of passwords in the database.
3. **Pipeline Verification:** Conducted a live demonstration of the CI/CD pipeline. A code push was shown triggering a Jenkins build, which subsequently deployed the updated Flask application to the EC2 instance automatically.

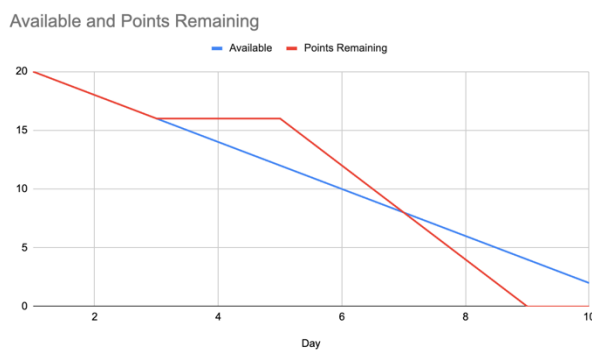
4. **Scope Confirmation:** Confirmed the successful inclusion and basic backend functionality of the Journey Planner endpoint, finalizing the open discussion from Sprint 1.

6.4 Sprint 3 — Additional Features & Frontend Implementation

Implemented Features and Design Decisions

Three AI chat features were implemented using the Langchain framework and the Aliyun Qwen API, with chat history persisted in new database tables and displayed on login. The React frontend was updated to show available bike and stand counts on station hover, and data is now pre-fetched on page load to eliminate display latency. The database was migrated from a local instance to Amazon RDS to overcome EC2 memory constraints and improve query performance. The CI/CD pipeline was verified end-to-end with Docker containerisation confirming a stable, reproducible deployment process.

Burndown Chart



This 10-day burndown chart tracks an initial 20 story points. Despite a complete progress stall between Days 3 and 5 that put them behind the ideal timeline, the team sharply accelerated their work. They ultimately recovered and finished all tasks a day early on Day 9.

Sprint Review

1. **AI Chat Features:** Successfully implemented the three core AI features using the Langchain framework and Aliyun API. Demonstrating chat history storage and LLM memory through new database tables, with chat history displayed upon user login.
2. **Backend Completion:** The Flask backend application is nearing completion, successfully returning the required data, confirmed through Postman testing.
3. **Frontend Data Display:** Updated the frontend to display available bike and stand data upon hovering over station icons. Data requests are now automatically sent on website load to eliminate display latency.
4. **CI/CD & Deployment:** Verified the continuous integration and deployment process using Jenkins. Confirmed the use of Docker and containerization for a professional, normalized deployment process, keeping the production application updated.

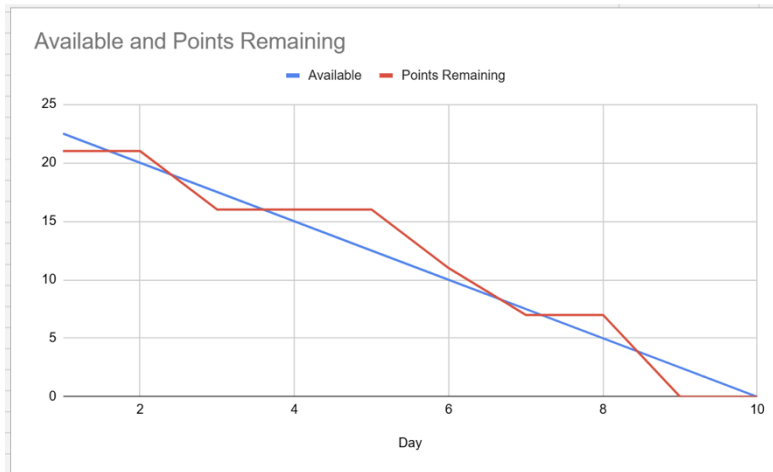
6.5 Sprint 4 — Machine Learning Integration & System Finalisation

Implemented Features and Design Decisions

A Random Forest model was trained on historical station and weather data, serialised as a .pkl file, and integrated into the Flask backend via a prediction service that preloads the model at startup. The React frontend was updated to fetch and render predictions dynamically, with a brief cold-start latency on the first request acknowledged and documented. UI styling was refined for consistency across all pages.

Backend test coverage reached 97% with pytest, and the CI/CD pipeline was updated to download ML artefacts from Hugging Face Hub during the build stage.

Burndown Chart



The Burndown chart illustrates a steady progression with a few plateaus, reflecting the complexity of our final integration tasks. Our "Points Remaining" line stayed slightly above the "Available" trend line during the middle of the sprint (Days 3-5) as the team was deeply involved in fine-tuning the machine learning algorithms and resolving frontend-backend API formatting issues. However, following a steep drop around Day 7, we regained our high velocity and successfully reached zero remaining tasks on Day 9, one day before the sprint deadline. This efficient completion allowed us to dedicate the final day strictly to code refactoring, ensuring UI consistency, and compiling our final project report.

Sprint Review

- Application Demonstration:** Presented the fully functional web application, showcasing the seamless integration between the React frontend and Flask backend. We demonstrated the refined UI styling, highlighting how the consistent design provides a smooth user experience across all pages.
- Machine Learning Verification:** Demonstrated the live predictive feature powered by our exported `.pkl` model. We showed how the frontend fetches data from the backend API and dynamically renders the prediction results. We also explained the slight "cold start" latency during the first click and proved the fast, seamless performance of all subsequent requests.
- Quality Assurance & Next Steps:** Highlighted our achievement of 97% unit test coverage, proving the stability and reliability of our codebase. Since all core development is now complete, we discussed our strategy for the final 2-week phase, agreeing to focus strictly on code refactoring, UI/UX beautification, and compiling the final project report.